

Desarrolla una **aplicación en Java** con los siguientes elementos

1. Clase base **Bicicleta**

Crea una jerarquía de bicis:

- Clase **Bicicleta** :
 - Atributos:
 - `velocidades: int` .
 - `pulgadas: int` .
 - `peso: int` .
 - `tamano: tan solo podrá tener los valores de ( "S", "M", "L" )` .
 - `estado: boolean`
 - `precio: int`
 - Métodos:
 - Constructores necesarios.
 - `mostrarDatos()` : muestra por consola todos los atributos.
 - `reparar()` : retorna double. no se sabe como se repara la bicicleta
 - Getters y setters.
- Crea 3 subclases :
 - **BicicletaMTB**
 - **BicicletaCarretera**
 - **BicicletaPaseo**

En cada subclase

- Añade **un método específico**:
 - **BicicletaMTB** : `hacerCaballito()` , muestre un mensaje por consola indicando que la MTB está haciendo el caballito.
 - **BicicletaCarretera** : `ponerModoSprint()` , muestre un mensaje indicando que entra en modo sprint e incrementa el numero de velocidades en 1 y reduciendo el peso en 25%.
 - **BicicletaPaseo** : `ponerCesta()` , muestre un mensaje indicando que se ha colocado una cesta, se reduce el numero de velocidades en 1 y se aumenta el paso en 25%.
- Cada una de las bicis se repara de la siguiente forma
 - MTB: Suma 2 a `velocidades` y retorna un número `double` aleatorio entre 100 y 200,
 - Carretera: Suma 1 a `velocidades` y 2 a `pulgadas y retorna` y retorna un número `int` aleatorio entre 50 y 100. Devuelve ese coste.
 - Paseo: retorna un número aleatorio entre 50 y 100.

2. Clase **Tienda** y clase **Almacen**

Crea la clase **Tienda** con:

- Atributos:

- `almacen: Almacen`.
- `bicicletasStock: ArrayList<Bicicleta>`.
- `caja: double`.
- Métodos:
 - `construirAlmacen`: crea el almacén.
 - `agregarBici`: añade la bici al stock **solo si hay plazas**; si no, lanza una excepción.
 - `agregarBicicletaReparar`: añade la bici a `bicicletasReparar`
 - `venderBicicleta`: vende la bici de `bicicletasStock` en una posición, muestra sus datos y suma a `caja` un importe. Una vez la bici se vende, se elimina de la lista
 - `repararBicis`: reparaá todas las bicis que están en el almacén. Además agregará una cantidad entre 50 - 100 a la caja de la tienda. Una vez la bici queda reparada, se elimina
- Clase `Almacen`
 - Atributos:
 - `numeroPlazas: int`
 - `bicisReparar: ArrayList`
 - Métodos:
 - aquellos que consideres necesarios
 - getters/setters y lógica para controlar plazas libres/ocupadas.

Regla: solo se admite una bici si el almacén tiene plazas libres. Si no hay plazas, se lanza una excepción personalizada.

3. Clase `Entrada`

En la clase `Entrada` (con `main`) realiza:

1. Crea una `Tienda` y un `Almacen` con 4 **plazas**.
2. Crea y agrega 5 **bicicletas al stock** (pueden ser de cualquier subclase).
3. Lleva a la tienda estas bicis concretas, para reparar :
 - **MTB de montaña** (`BicicletaMTB`):
 - 27 velocidades, 29 pulgadas, tamaño `L`, tipo `"MTB"`.
 - **Bici de paseo** (`BicicletaPaseo`):
 - 2 velocidades, 27 pulgadas, tamaño `M`, tipo `"paseo"`.
 - **Bici carretera** (`BicicletaCarretera`):
 - 8 velocidades, 24 pulgadas, tamaño `S`, tipo `"carretera"`.

Cada vez que intentes **admitir** una bici en la tienda, comprueba si hay plazas en el almacén; si no, lanza y captura la excepción mostrando un mensaje.

1. Muestra los datos de todas las bicis en `bicicletasStock`.
2. Repara todas las bicis pendientes, muestra sus datos y suma a `caja` el coste de cada reparación.
3. Genera un número aleatorio entre **1 y 20** e intenta vender la bici en esa posición del `bicicletasStock`.
 - Si la posición no existe, captura la excepción y muestra un mensaje